One must execute after the other, and the default behavior is that `TempData` population takes place *after* model binding. So any value assigned to the property by model binding will be overwritten by whatever is in `TempData`, which is usually nothing. Consequently, you will make the assignment manually.

Your fairly simple renaming logic removes spaces and hyphens from the city name and then uses the lowercase version as a filename. Then the file is saved in the cities folder, located in wwwroot\images, alongside the Flags folder you created in the last chapter. You will need to create that folder manually before saving files to it. After the file is saved, you redirect the user to a page that doesn't exist yet.

Before you create that page, you need to add the form to the Create.cshtml file. Listing 6.27 shows the code for the upload form. You should be familiar with most of this by now. The only point of real note is the addition of the `enctype` attribute on the form tag helper, specifying how the form's data should be encoded. This is set to `multipart/ form-data`, which is one of the three requirements for successful form uploading. If you omit this, the encoding will default to `application/x-www-form-urlencoded`, resulting in only the filename being posted to the server. You have applied an `accept` attribute to the file input. Its value is set to the file extension the upload should be restricted to. User agents that support this attribute will apply a filter to the available files the user can select from to prevent the wrong type being uploaded. This could be considered an additional means of client validation, but you should not rely on it.

Listing 6.27 Create City form

```
@page
@model CityBreaks.Pages.CityManager.CreateModel
@{
}
```